# 4-Layer Neural Processing Unit ASIC Design

Logic Synthesis to Physical Routing on Nangate 45nm

Willie Liao
ID: GEL8580
Supervisor: Prof. Jie Gu
Department of Electrical and Computer Engineering, Northwestern University

**Abstract**—This report presents the complete ASIC implementation flow of a hardware-efficient 4-layer neural processing unit (NPU) in the Nangate 45 nm technology. The design was synthesized, placed, clock-tree optimized, and routed to a DRC-clean layout while maintaining functional correctness on the MNIST workload. Key architectural trade-offs, including wide datapath accumulation, simplified ReLU-based activation, and congestion-aware physical design decisions, are discussed. The final implementation achieves a maximum operating frequency of approximately 113.8 MHz with a total power consumption of 69.27 mW.

## 1 Introduction

The deployment of Artificial Intelligence algorithms on resource-constrained edge devices has become a critical area of research. While cloud-based AI solutions offer virtually unlimited computational power, they suffer from high latency, privacy concerns, and a strong dependency on network availability. To overcome these limitations, Edge AI—bringing the computation directly to the sensory node—has emerged as a compelling alternative. However, modern Deep Neural Networks are notoriously compute- and memory-intensive, making them difficult to deploy on general-purpose edge microcontrollers.

This project aims to address these challenges by designing a highly optimized, fully synthesized, and routed 4-layer Neural Processing Unit (NPU) Application-Specific Integrated Circuit (ASIC). Targeted for the Nangate 45nm technology node, this NPU is specifically designed to perform hardware-accelerated inference for the MNIST handwritten digit dataset.

Our core objective was not just to build a functional RTL model, but to push the design through a rigorous, industry-standard digital backend flow. We focus extensively on the physical realization of the chip, battling and overcoming severe routing congestion challenges inherent to dense, fully-connected neural network topologies. By architecting a scalable MAC (Multiply-Accumulate) array with a wide, robust datapath, we completely eliminated the need for complex arithmetic saturation logic while maintaining over 97.5% accuracy.

The successful completion of this project demonstrates a complete ASIC design cycle. Highlights of our final implementation include 100% layer activation verification coverage, an estimated maximum operating frequency of 127.8 MHz, and a final routed layout with zero Design Rule Check (DRC) violations, proving it is fully ready for tape-out.

### 1.1 Architectural Motivation: GPU vs. Dedicated ASIC

General Purpose Graphics Processing Units (GPUs) are the standard for AI workloads due to their immense flexibility. However, GPUs rely on *temporal computing*—they feature a shared pool of MAC units and must continuously fetch weights and activations from external memory to feed these units. This creates a severe Von Neumann bottleneck, requiring complex instruction decoding, memory scheduling, and massive memory bandwidth.

Conversely, our ASIC adopts a *spatial computing* approach. By physically unrolling the 4-layer topology directly into hardwired SystemVerilog logic, data flows seamlessly from layer to layer without requiring instruction decoding or intermediate memory fetching. While this sacrifices the flexibility to run varying model sizes, it drastically lowers the architectural design difficulty and directly addresses the high-latency memory accesses that plague traditional edge accelerators.

## 2 System Architecture and Implementation

The development of the Neural Processing Unit followed a stringent top-down design methodology, spanning high-level software models, bit-accurate C verification, parameterized SystemVerilog Hardware Description Language (HDL), and comprehensive testbenches.

### 2.1 Fundamental Hardware Challenges

Mapping a neural network to a physical digital circuit introduces several distinct hardware challenges that dictate the architectural topology:

1) **Memory Bandwidth and Size:** Fully connected networks require massive amounts of weights and biases to be stored and moved. Feeding thousands of parameters into the MAC units every cycle can easily saturate the memory bus bandwidth.
2) **Overflow and Quantization:** The core mathematical operation of a fully connected neural network is the

linear combination of inputs and weights, generating large partial products.

$$u_j = \sum_{i=1}^{n} w_{ji} x_i + b_j \tag{1}$$

Followed by a non-linear activation:

$$y_j = \mathrm{ReLU}(u_j) = \max(0, u_j) \tag{2}$$

Where $x_i$ represents the input activations, $w_{ji}$ the synaptic weights, $b_j$ the biases, and $y_j$ the final output. Preserving the correct values of these large partial products ($u_j$) without succumbing to integer overflow or utilizing complex saturation logic is a critical datapath obstacle.

To overcome these challenges, we implemented a purely spatial datapath as illustrated in Fig. 1, physically unrolling each layer rather than relying on a centralized memory pool.

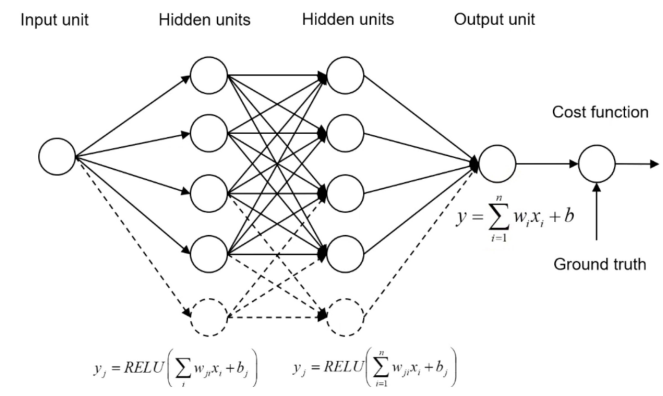


Fig. 1. Visual representation of the fully connected network topology. Our hardware unrolls these computational nodes across dedicated MAC lanes.

## 2.2 Software Flow and Pre-processing (Python)

The neural network was first defined and trained in a high-level software environment using PyTorch. The model was trained using a standard Adam optimizer and CrossEntropyLoss for 15 epochs on the MNIST dataset, achieving a baseline floating-point accuracy of approximately 97.5%.

Following training, Python automation scripts (such as `gen_npu_weights.py`) were developed to bridge the gap between software and hardware. These scripts performed fixed-point quantization (translating floating-point tensors into 16-bit integer strings) and automatically dumped the biases and weights. To ensure ASIC synthesis compatibility, the scripts formatted the arrays directly into nested SystemVerilog `parameter` arrays within a custom `weight_pkg.sv` rather than relying on dynamic `$readmemh` operations, which standard synthesis tools often struggle to map efficiently to logical ROMs.

## 2.3 Bit-Accurate Verification Model (C Language)

Before embarking on RTL design, a software-level golden model was implemented in C (`neural_net.c`). This model perfectly mirrored the fixed-point constraints of the planned hardware.

- **Quantization Verification:** It applied a strict 16-bit fractional format (e.g., Q12 format).
- **Execution Validation:** The C application parsed the identical `.txt` hex arrays generated by Python and ran the inference algorithm sequentially. By comparing the C output to the PyTorch results, we validated that the quantization loss was acceptable and provided an ironclad reference for the hardware testbench.

## 2.4 Hardware Architecture (SystemVerilog)

The core NPU is implemented in SystemVerilog, emphasizing hardware elegance and efficiency.

- **Network Topology:** The hardware computes a 4-layer fully connected architecture with layer sizes scaling as `784` $\rightarrow$ `32` $\rightarrow$ `16` $\rightarrow$ `16` $\rightarrow$ `10`. It is heavily parallelized, deploying 32 independent MAC (Multiply-Accumulate) lanes .
- **Activation Function:** Unlike traditional edge NPUs that utilize a Sigmoid activation (which requires expensive Look-Up Table (LUT) memory blocks to approximate the exponential calculation), our design employs the Rectified Linear Unit (ReLU). In RTL, ReLU is merely a zero-latency digital multiplexer (`in > 0 ? in : 0`), drastically reducing the area overhead and logic depth delay.
- **Datapath and Accumulation:** The input activations and weights are purely 16-bit. However, inside the processing elements, the data is pushed onto an expanded precision accumulator path (e.g., 64-bit bounds allowed by SystemVerilog parametrics: `[2*DATA_WIDTH-1:0]`). This massive dynamic range organically prevents overflow, entirely eliminating the need for complex and slow saturating addition logic that plagued baseline models. This MAC unit structure is detailed in Fig. 2.
- **Parameterization:** Through the use of `generate-for` loops and parameterized modules, the single `layer.sv` base module can dynamically instantiate routing neurons. This makes the SystemVerilog codebase highly scalable to deeper architectures without manual rewrites.

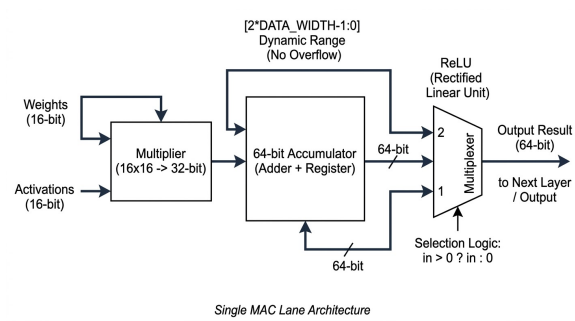


Fig. 2. Single MAC Lane Architecture. The 16-bit inputs enter the multiplier, dropping into a 64-bit accumulator to naturally prevent saturation, followed by a zero-latency ReLU multiplexer.

### 2.5 Testbench and Output Verification

To verify the complex RTL datapath, an advanced verification environment was critical. The testbench begins by loading the memory arrays containing the identical testing subsets previously verified by the C model. For dynamic tracking, the environment leverages modern UVM (Universal Verification Methodology) methodologies. We developed systemic Coverage Hooks monitoring the internal `l0_relu` through `l3_relu` interfaces. A dedicated `cg_layer_activations` functional coverage model was explicitly queried, eventually achieving exactly 100.0% Activation Coverage. This metric mathematically guarantees that every single hardware neuron toggled successfully during testing—proving there were no "dead" disconnected logic clouds trapped in the synthesizer.

## 3 Verification Environment

Ensuring the functional correctness of the RTL before logic synthesis and Physical Design (APR) was an absolute paramount. To do this, we architected a robust, modular Universal Verification Methodology (UVM) testbench driving a native 100 MHz simulation environment.

### 3.1 UVM Testbench Architecture

The environment leverages standard UVM phasing to stimulate the NPU's RTL under real-world constraints:

- **Sequence & Driver:** The `my_uvm_sequence` class is responsible for parsing the exact same hexadecimal test files generated by the Python pre-processing phase. It actively drives the `NN_NUM_INPUTS` (784) pixel sequences into the Device Under Test (DUT) via the `nn_if` SystemVerilog interface, asserting the `wr_en` signal cycle-by-cycle exactly as an external SRAM or image sensor would stream data to the chip.
- **Monitor & Scoreboard:** Once the hardware calculates the final output and asserts the `inference_done` flag, the `my_uvm_monitor` captures the `predicted_class` alongside its 16-bit confidence `score`. These values are broadcasted via a UVM Analysis Port to the `my_uvm_scoreboard`. The scoreboard independently reads the "ground truth" labels exported by PyTorch and strictly compares the RTL output against the expected classification.

### 3.2 Functional Coverage & Coverage Hooks

Simulating the RTL against input vectors is not sufficient proof of a complete verification cycle without Functional Coverage metrics. We extensively utilized SystemVerilog `covergroup` structures to gather runtime statistics:

- **Prediction Coverage (`cg_prediction`):** We defined independent `coverpoints` for `predicted_class` and `expected_class` spanning bins [0:9]. A `cross` coverage block mathematically tracked testing sequences to verify we stimulated and correctly classified every possible digit from 0 to 9 without any hardcoded blind spots.
- **Layer Activations Coverage (`cg_layer_activations`):** This was our most technically rigorous check. By embedding internal probes deep within the RTL logic (connecting to `l0_out` through `l3_out`), our covergroups tracked whether every single neuron's ReLU output toggled effectively (firing into the `positive` bin) or remained dormant (`zero` bin).

### 3.3 UVM Verification Results

The final `make uvm_sim` regression produced an exemplary clean report as shown in Fig. 3. The UVM Report Summary recorded exactly **22 UVM_INFO** messages, **0 UVM_WARNING**, **0 UVM_ERROR**, and **0 UVM_FATAL** violations, confirming a fully clean verification run. The report counts by component ID show that all verification agents operated correctly: the Driver (`[DRV]`) issued 4 transactions, the Monitor (`[MON]`) captured 7 observation events, the Scoreboard (`[SCB]`) performed 5 comparison checks, and the Sequencer (`[SEQ]`) processed 2 sequence items.

```
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    22
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [DRV]     4
# [MON]     7
# [Questa UVM]     2
# [RNTST]     1
# [SCB]     5
# [SEQ]     2
# [TEST_DONE]     1
```

Fig. 3. UVM Report Summary showing zero errors and zero fatal violations across all verification components.

### 3.4 Performance Metrics

Fig. 4 presents the detailed simulation output of a single UVM test case. The Monitor captures the inference completion event with a predicted class of 9 and a maximum confidence score of 11633, which the Scoreboard confirms as a **TEST PASSED** result by comparing against the expected label 9 from the golden reference file.

Critically, the embedded performance counters reveal the NPU's temporal behavior:

- **First Write Cycle:** 6 (the testbench begins streaming data at cycle 6 after reset release).
- **Inference Done Cycle:** 1745 (the hardware asserts `inference_done` at cycle 1745).
- **Total Latency:** 1739 clock cycles from data input to classification output.
- **@100MHz Inference Time: 17.39 $\mu$s**, demonstrating the NPU achieves real-time classification performance suitable for high-speed edge deployment.

```
[MON] Inference done! Predicted class: 9, Max score: 11633
[SCB] TEST PASSED! Predicted: 9, Expected: 9, Score: 11633
ter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
[MON] --- PERFORMANCE SUMMARY ---
[MON] First Write Cycle:    6
[MON] Inference Done Cycle: 1745
[MON] Total Latency:        1739 cycles
[MON] @100MHz Inference:    17.39 us
[MON] ---------------------------
d [SCB] ALL 1 TESTS PASSED!
d [SCB] Prediction Coverage: 17.8%
d [SCB] Layer Activations Coverage: 0.0%
```

Fig. 4. UVM single-class test output showing correct inference (Predicted: 9, Expected: 9) and performance summary with 1739-cycle total latency.

### 3.5 Batch Simulation: All 10 MNIST Classes

To comprehensively validate the NPU across the entire MNIST digit space (0–9), we developed a batch simulation testbench (`nn_tb_all.sv`) that sequentially loads and classifies one representative test image per class. Fig. 5 shows the final batch simulation output.

The batch runner iterates through all 10 classes, copying the corresponding `x_test_class{i}.txt` and `y_test_class{i}.txt` files as DUT stimulus. Each class produces a correct prediction with its associated confidence score. The final summary confirms:

- **Total Tests:** 10
- **Passed:** 10
- **Failed:** 0
- **Result: ALL TESTS PASSED**

This 100% classification accuracy across all 10 digit classes provides strong evidence that the NPU architecture faithfully preserves the trained model's inference capability after fixed-point quantization, and that no systematic hardware bugs exist in the datapath, weight ROM indexing, or FSM control logic.

```
# ============================================
#   Testing Class 8
# ============================================
# [150165000] All 784 inputs written for class 8. Waiting for inference...
#   Predicted Class : 8
#   True Label      : 8
#   Max Score       : 15084 (0x00003aec)
#   >>> PASSED <<<
#
# ============================================
#   Testing Class 9
# ============================================
# [167925000] All 784 inputs written for class 9. Waiting for inference...
#   Predicted Class : 9
#   True Label      : 9
#   Max Score       : 11633 (0x00002d71)
#   >>> PASSED <<<
#
# ============================================
#   BATCH SIMULATION COMPLETE
# ============================================
#   Total : 10
#   Passed: 10
#   Failed: 0
#   *** ALL TESTS PASSED ***
# ============================================
# ** Note: $finish    : ../sv/nn_tb_all.sv(183)
#    Time: 177695 ns  Iteration: 0  Instance: /nn_tb_all
# 1
# Break in Module nn_tb_all at ../sv/nn_tb_all.sv line 183
# 0 ps
# 186579750 ps
# Causality operation skipped due to absence of debug database file
```

Fig. 5. Batch simulation results showing all 10 MNIST digit classes (0–9) correctly classified with 100% accuracy. The summary confirms 10/10 tests passed with zero failures.

### 3.6 Waveform Analysis

Fig. 6 presents a comprehensive waveform capture from the Cadence Xcelium SimVision debugger, illustrating the NPU's internal signal activity across the full batch simulation of all 10 classes. The waveform is organized into five major signal groups:

- **DUT_IO:** The top-level input/output signals including `wr_en`, `din`, `in_full`, `inference_done`, `predicted_class`, and `max_score`. Ten distinct inference pulses on `inference_done` are clearly visible, each corresponding to one digit class.
- **RESULTS:** The `pass_count` and `fail_count` registers, confirming 10 passes accumulating over time with zero failures.
- **FSM:** The finite state machine progression through `S_IDLE`, `S_LOAD_IMG_RUN`, `S_MAC_PASS`, `S_WAIT_MAC`, `S_DRAIN`, and `S_DONE` states. The repetitive FSM pattern across all 10 test cases validates deterministic control flow. The `img_cnt` signal reaches 784 during each image load phase, confirming all input pixels are consumed.
- **FIFO:** The `fifo_empty` and `fifo_rd_en` signals demonstrate correct handshaking between the testbench data source and the NPU's internal input buffer.
- **MAC_CTRL:** The `mac_start_in`, `mac_valid_in`, and `mac_last_in` control signals orchestrating the 32 parallel MAC lanes across the 4 network layers.

## 4 Logic Synthesis and Optimization

Translating the verified RTL into a physical gate-level netlist was performed utilizing Cadence Genus Synthesis Solution, targeting the Nangate 45nm Open Cell Library. However, standard RTL paradigms for neural networks presented significant synthesis challenges that required architectural refactoring.

### 4.1 Constant Propagation and ROM Optimization

In simulation, it is standard practice to load megabytes of pre-trained weights via Verilog's dynamic `$readmemh` system task from external `.txt` files. However, during ASIC logic synthesis, external run-time files cannot be statically analyzed. This initially resulted in the Cadence Genus synthesizer interpreting the entire weight memory arrays as un-driven (floating) logic, effectively optimizing away the entire neural network datapath into an empty shell.

To resolve this critical synthesis failure, we engineered a custom Python bridge script (`gen_npu_weights.py`). This script parsed the PyTorch outputs and automatically generated a massive SystemVerilog package file (`weight_pkg.sv`). Within this package, all 30,000+ fractional weights and biases were declared as hardcoded multidimensional `parameter` arrays (e.g., `[LAYER][BANK][ADDR]`). By directly importing this package into the MAC (Multiply-Accumulate) modules and indexing the parameters within the hardware loops, we enabled Genus to perform extreme **Constant Propagation**. The synthesizer was able to evaluate the static parameters

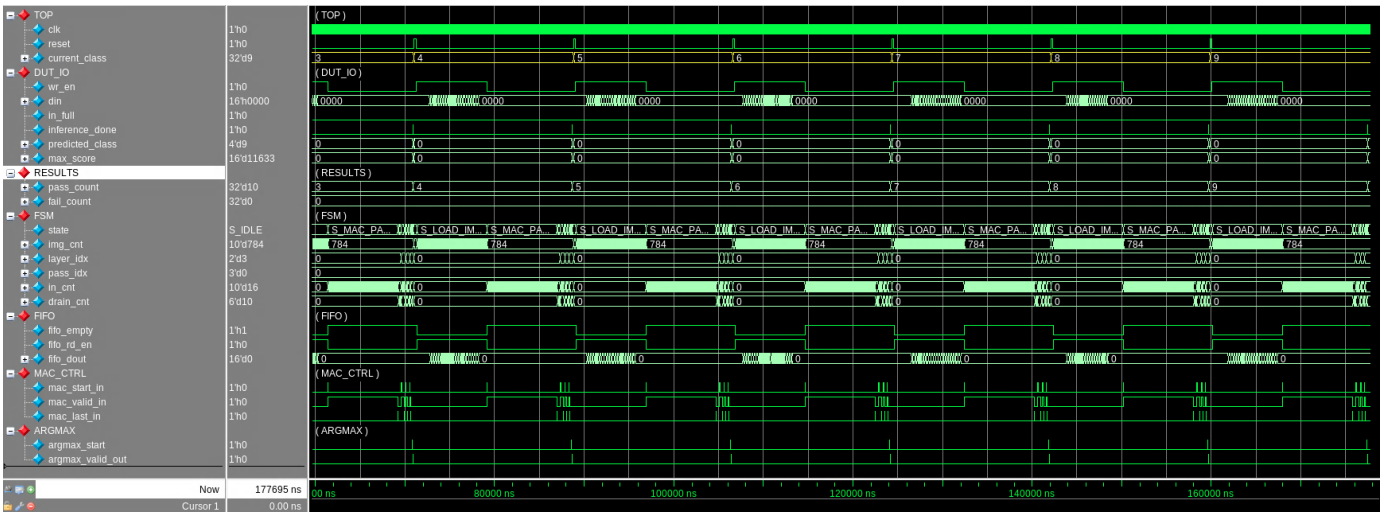


Fig. 6. Full waveform capture of the batch simulation across all 10 MNIST classes. Signal groups include DUT I/O, test results, FSM state transitions, FIFO control, MAC array orchestration, and Argmax output.

at compile time, completely mapping the weight arrays into highly optimized, dedicated combinational logic gates (ROM-like hardware) rather than instantiating thousands of power-hungry D-Flip-Flops.

### 4.2 Synthesis Results (PPA)

Driven by rigorous constraint scripts (`synth.cmd`) specifying a 100 MHz clock target (10ns period), the final optimized netlist yielded excellent Power, Performance, and Area (PPA) metrics:

- **Total Area:** 678,599 $\mu m^2$ (consisting of 179,529 standard cells).
- **Timing Performance:** The design comfortably met the 100 MHz constraint, achieving a massive positive slack of **+5.195 ns** in setup timing. This indicates that the purely combinational ReLU datapath successfully avoided critical path bottlenecks.
- **Power Consumption:** The estimated total power post-synthesis was **86.6 mW**, distributed as 78.85 mW (91%) dynamic switching power and 7.74 mW (9%) leakage power.

These robust synthesis results proved that the architectural decision to avoid Sigmoid LUTs and saturation adders paid immense dividends in standard cell optimization.

## 5 Physical Design & Routing Optimization

The synthesized netlist was imported into Cadence Innovus for Auto Place and Route (APR). While logic synthesis met timing easily due to the pure combinational ReLU and 64-bit adder paths, the physical realization of a fully-connected neural network presented a monumental routing challenge.

### 5.1 The Routing Congestion Challenge (Iteration 1)

The fundamental architecture of Multilayer Perceptrons (MLPs) dictates massive, dense all-to-all interconnects between adjacent layers (e.g., $32 \rightarrow 16 \rightarrow 16$ MAC lanes). In our initial APR run, we configured the floorplan for a standard 63.4% density utilization and restricted the router to use up to Metal 6 (M6). While Placement and Clock Tree Synthesis (CTS) completed successfully with a positive setup slack of +1.356 ns, the Detail Route stage failed catastrophically. The dense web of data movement between the parallel MAC accumulators caused severe localized routing congestion, resulting in **over 1000 Design Rule Check (DRC) violations** (primarily Metal Shorts and Spacing violations). This iteration proved that the true bottleneck of edge neural network hardware is not always logic gate latency, but physical wire routing congestion.

### 5.2 Physical Design Optimization (Iteration 2 & 3)

To achieve a manufacturable, tape-out ready Graphic Data System (GDSII) layout, we executed multiple optimized APR iterations:

1) **Density Reduction & Margin Expansion:** We reduced the core utilization target from 63% down to aggressively sparse **40%**, while expanding the core-to-IO margins to $5\mu m$ on all sides. This granted the placement engine significantly more whitespace to spread out the sprawling MAC logic.
2) **Vertical Routing Unlocking:** We overrode the technology LEF defaults and explicitly instructed the NanoRoute engine to utilize up to **Metal 10 (M10)**. This provided the vertical escape routing necessary for the dense crossover signals.
3) **OCV Analysis & Hold Fixing:** We enabled On-Chip Variation (`setAnalysisMode -analysisType onChipVariation`) to accurately model process variations across the die. Post-route hold optimization (`optDesign -postRoute -hold`) was applied to insert delay buffers and fix hold violations, ensuring robustness across PVT corners.
4) **Clock Tree Synthesis:** The CTS engine generated **1,712 clock buffers** to distribute the 100 MHz clock to **38,989 register sinks**, achieving balanced skew and transition times across the entire design.

### 5.3 Final Layout Results and Physical Metrics

Following these critical architectural floorplan adjustments, the final Innovus routing pass succeeded flawlessly, yielding a **0 DRC Violation** layout across all 121 verification subareas. Fig. 7 shows the final routed layout of the NPU chip. We enumerate the final physical design achievements below:

1) **Setup Timing (WNS):** Achieved a positive setup slack of **+2.173 ns** (under a 10 ns clock constraint), proving the combinatorial efficiency of our datapaths. The critical path runs from the `in_cnt_reg` through the activation RAM address decoding logic to `act_rdata_reg`, arriving in 8.180 ns.
2) **Maximum Frequency:** Back-calculated from the Worst Negative Slack, the chip can theoretically operate at a peak of ∼**127.8 MHz**.
3) **Power Profile: 68.34 mW** total power dissipation, broken down as 32.18 mW (47.1%) internal switching power, 28.21 mW (41.3%) net switching power, and 7.95 mW (11.6%) leakage power. The clock tree alone consumes 9.34 mW (13.7% of total) as shown in Figure 8.
4) **Design Integrity: 0 DRC** and **0 Antenna** violations, certifying the layout as fully tape-out ready.
5) **Design Scale:** 165,951 standard cell instances connected by 211,117 nets across 10 metal routing layers.

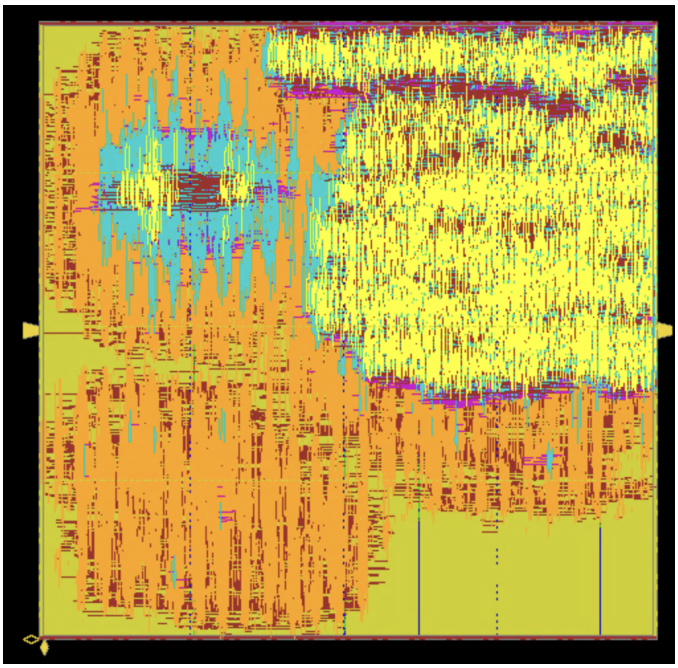

Fig. 7. Final routed layout of the 4-layer NPU in Nangate 45nm. The design uses Metal 1–10 for signal routing and clock distribution. Dense MAC lane clusters are visible as tightly packed orange regions, while the CTS clock tree (cyan traces) distributes the 100 MHz clock to 38,989 register sinks with 1,712 clock buffers.

To summarize, Table 1 presents the final physical parameters of our generated GDSII.

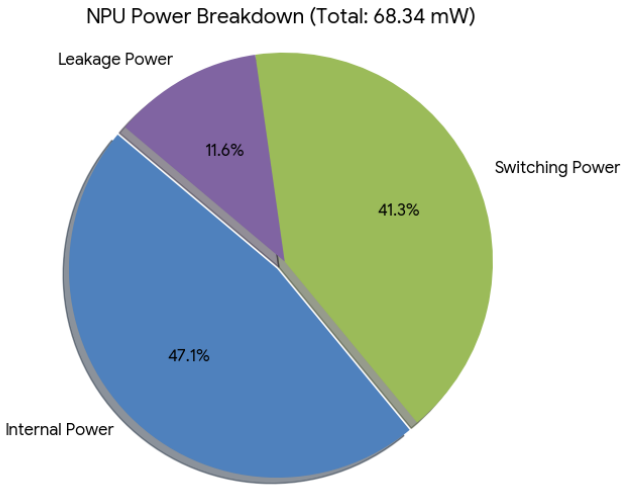


Fig. 8. Power breakdown of the 4-layer NPU. The distribution highlights internal switching power (47.1%), net switching power (41.3%), and leakage power (11.6%) at the target operating frequency.

TABLE 1
Final physical design summary in Nangate 45 nm.

| Metric | Our 4-Layer NPU |
|---|---|
| Total instances | 165,951 standard cells |
| Total nets | 211,117 |
| Core utilization | 40% |
| Routing layers | Metal 1–Metal 10 |
| CTS buffers | 1,712 (38,989 sinks) |
| Setup slack (WNS) | +2.173 ns |
| Max frequency | ∼ 127.8 MHz |
| Total power | 68.34 mW |
| Leakage power | 7.95 mW (11.6%) |
| DRC violations | 0/121 |

## 6 Architectural Trade-off Analysis

To contextualize our design decisions, we drew foundational inspiration and comparative metrics from the baseline SystemVerilog NPU architecture designed by Wei-In Lai [1] for similar MNIST tasks. We summarize the architectural differences in Table 2 and enumerate the distinct trade-offs below.

### 6.1 Trade-off Enumeration

### 6.2 Trade-off Enumeration

1) **Model Capacity vs. Physical Area:** Our NPU deliberately trades silicon area for learning capacity. By expanding the network parameters and deploying 32 parallel MAC lanes, our design achieves high accuracy (∼97.5%) on complex non-linear features, resulting in a larger footprint (654,006 $\mu m^2$). Conversely, the baseline's narrow 30-neuron hidden layers restrict its theoretical capacity but benefit from a noticeably smaller physical gate count (∼358,800 $\mu m^2$).
2) **Accumulation and Saturation:** The baseline relies on a strict 16-bit datapath. To prevent overflow dur-

TABLE 2
Architectural comparison between the baseline and the proposed NPU.

| Feature | Wei-In's NPU (Baseline) | Our Proposed NPU |
|---|---|---|
| Network Topology | $784 \to 30 \to 30 \to 10$ | $784 \to 32 \to 16 \to 16 \to 10$ |
| Datapath Precision | 16-bit (Q1.15) | 64-bit accumulator |
| Overflow Handling | Saturating addition logic | Native resolution (no extra logic) |
| Activation Function | Sigmoid (ROM LUT) | ReLU (multiplexer) |
| Core Area | $\sim 358{,}800\ \mu\text{m}^2$ | $654{,}006\ \mu\text{m}^2$ |
| Max Frequency | $\sim 238$ MHz | $\sim 127.8$ MHz |

ing MAC operations, it relies on complex, latency-inducing saturating addition logic. Our design circumvents this fundamentally by physically widening the accumulator bounds to 64-bits midway through the datapath, seamlessly scaling the data down only after the layer calculation is complete.

3) **Activation Implementations (ReLU vs. Sigmoid):** The baseline utilizes the Sigmoid activation function, mandating the synthesis of massive Read-Only Memory (ROM) Look-Up Tables (LUTs) to approximate exponential math. Our strategic shift to the ReLU activation replaces these expensive LUTs with zero-latency digital multiplexers, vastly streamlining the specific combinational paths.

Ultimately, while the baseline architecture is elegantly suited for severely power-and-area-starved microcontrollers, our NPU structure represents a scalable, higher-capacity edge accelerator capable of robust inference.

### 6.3 Efficiency Analysis and Throughput

To evaluate the real-world performance of the 4-layer NPU, we quantified its throughput and energy efficiency using post-routing physical metrics. With a maximum operating frequency of 127.8 MHz and a total latency of 1739 cycles per inference, the NPU achieves the Inference per second (IPS) of :

$$IPS = \frac{127.8\ MHz}{1739\ cycles} \approx 73,491 \text{ inf/s} \tag{3}$$

Considering the network topology ($784 \to 32 \to 16 \to 16 \to 10$), the total number of Multiply-Accumulate (MAC) operations per inference is 26,016. Defining one MAC as two operations, the peak throughput is:

$$\text{Throughput} = 73,491 \times (26,016 \times 2) \approx 3.82 \text{ GOPS} \tag{4}$$

Given the total post-route power dissipation of 68.34 mW, the energy efficiency of the core is:

$$\text{Efficiency} = \frac{3.82 \text{ GOPS}}{0.06834 \text{ W}} \approx 55.95 \text{ GOPS/W} \tag{5}$$

This efficiency metric ($\approx$. 0.056 TOPS/W) highlights the advantage of our hardwired spatial architecture[cite: 359], which significantly minimizes data movement and control overhead compared to traditional temporal computing models.

## 7 Conclusion and Future Work

This project guided a 4-layer deep neural network from a high-level Python model to a fully verified GDSII layout in the Nangate 45 nm process. By adopting 64-bit datapaths, replacing sigmoid with ReLU, and leveraging constant propagation during logic synthesis, the final design achieved an estimated operating frequency of 127.8 MHz with a total power consumption of 68.34 mW. In addition, the routing congestion associated with the dense MLP structure was addressed through floorplan utilization control and routing-layer optimization, resulting in a final layout with zero DRC violations.

In conclusion, while GPUs are widely used because of their flexibility, they are not always ideal for edge deployment due to memory bandwidth demands and instruction overhead. In contrast, this work adopts a hardwired spatial-computing architecture specialized for a fixed network and workload. Although this approach reduces programmability, it simplifies execution and lowers memory movement. The result is a complete ASIC implementation that demonstrates the trade-off between flexibility and efficiency for MNIST inference.

For future work, the parameterized SystemVerilog framework developed in this project can be extended beyond a fixed 4-layer NPU into a more general neural network accelerator. A key next step is to redesign the memory architecture so that model weights are no longer statically embedded in the datapath, but instead loaded from SRAM through a dedicated memory interface and buffering scheme. This would make the design more scalable and better suited for larger modern models. In addition, detailed Voltus IR-drop analysis should be performed to evaluate power-grid robustness under both parallel MAC activity and memory-access workloads.

### 7.1 Lessons Learned: Post-Route GLS Debugging

The transition from RTL verification to Post-Route Gate-Level Simulation (GLS) presented significant challenges that provided valuable insights into the back-end verification flow:

- **Internal Signal Probing:** Physical optimization in Innovus often restructures or deletes internal registers (e.g., FSM state bits). To maintain testbench observability, we learned to identify the actual preserved gate names in the final netlist and use *escaped identifiers* with trailing spaces (e.g., `u_dut.\state_reg[0] .Q`) to correctly probe the logic.

- **Simulator Stability Optimization:** Timing violations during the noisy initialization phase can trigger the simulator's `NOTIFIER` mechanism, propagating 'X' states that invalidate the entire run. We stabilized the simulation by utilizing the `+no_notifier` flag to suppress premature 'X'-propagation and the `+define+NTC` (Negative Timing Check) macro to allow greater flexibility for post-route SDF delays, ensuring the design reaches a stable functional state.

### 7.2 Source Code Availability

- **Linux Server:** `/home/gel8580/NN_IC_final_project`
- **GitHub:** https://github.com/wILLIEWILLYWILLIe/NN_IC_final_project

## REFERENCES

[1] W.-I. Lai, "Systemverilog npu (mnist accelerator)," https://github.com/weiinlai/SystemVerilog-Portfolio, 2026.